Developer's Manual

StudyFlax: Time Management for Higher Education

Team Flax-10
Aishah Ahmed
Eric Almeida
Zerun (Harry) Chen
Cate Tay (Cate) Chew
Indixora Jozal
Kausar Khan
Hamed Latif
Dian Mahmood
Binuli Nevanma Markawitage

2026-03-29

Overview

StudyFlax is a web-based academic time management platform built for students and teachers at King's College London. It allows students to organise their schedules, track deadlines, and manage their academic workload in one place. Teachers can create modules, set deadlines, and assign work to their students, whilst the admin oversees and manages the system as a whole.

The system consists of two software components:

- **Backend** — A Django REST API developed primarily with Python 3.12 in the Nix environment and backed by a PostgreSQL database.
- **Frontend** — A React (Vite) single-page application.

The deployed application is available at: https://flax-10-production.up.railway.app/

Prerequisites

The following tools must be installed before setting up the development environment. Installation of the operating system itself is assumed.

- **Nix Package Manager** (v2.33.2 or later)
  https://nixos.org/download/#download-nix
- **Git**
  https://git-scm.com/downloads
- **Python** (v3.12 recommended for manual local setup)
  https://www.python.org/downloads/
- **Node.js** (v20 recommended)
  https://nodejs.org/
- **PostgreSQL** (only required for manual setup without Nix)
  https://www.postgresql.org/download/

Cloning the Repository

Clone the repository and navigate into it:

```
git clone https://github.com/hoxtl/Flax-10.git
cd Flax-10
```

Setting Up the Development Environment with Nix

This project uses a Nix flake to provide a reproducible development environment. The Nix shell itself only provides the required tools and prints the available entrypoints. Project setup and common tasks are then run explicitly using the Nix packages defined in `flake.nix`.

- `nix develop` enters the shell and exposes Python, Node.js, Git, and PostgreSQL tooling.
- `nix run .#init` creates the project virtual environment, installs Python and Node dependencies, prepares a project-local PostgreSQL instance, applies migrations, and seeds the database.
- `nix run .#run` starts the backend and frontend development servers.
- `nix run .#tests` runs backend and frontend automated tests and writes coverage reports to `./coverage/`.
- `nix run .#seed` and `nix run .#unseed` manage seeded data without reinitialising the whole project.

Linux and macOS

1. Navigate to the root of the repository:

   ```
   cd /path/to/Flax-10
   ```

2. Enter the Nix development shell:

   ```
   nix --extra-experimental-features 'nix-command flakes' develop
   ```

3. **Optional — enable experimental features permanently.** Add the following line to `/etc/nix/nix.conf` to avoid passing the flags on every invocation:

   ```
   experimental-features = nix-command flakes
   ```

4. Initialise the repository:

   ```
   nix run .#init
   ```

5. Start the development servers when required:

   ```
   nix run .#run
   ```

6. Run the full automated test suite when required:

   ```
   nix run .#tests
   ```

Re-entering the Nix Shell in Future Sessions

Each time you wish to return to the development environment, run the following from the root of the repository:

```
nix --extra-experimental-features 'nix-command flakes' develop
```

Database Setup

Automated Setup via Nix (Recommended)

The recommended Nix workflow does not require a separately installed PostgreSQL server. The `nix run .#init`, `nix run .#run`, `nix run .#tests`, `nix run .#seed`, and `nix run .#unseed` commands start a project-local PostgreSQL cluster in `.postgres/data`, create the `flax10` database, and export the required `POSTGRES_*` variables automatically.

Manual PostgreSQL Setup

If you are setting up the project without Nix, install and configure PostgreSQL first.

1. Create a PostgreSQL database and user matching the Django defaults in `settings.py`, or set the corresponding environment variables manually:

   ```
   CREATE USER postgres WITH PASSWORD '1234';
   CREATE DATABASE postgres OWNER postgres;
   ```

2. Alternatively, export custom connection variables before running Django commands:

   ```
   export POSTGRES_DB=postgres
   export POSTGRES_USER=postgres
   export POSTGRES_PASSWORD=1234
   export POSTGRES_HOST=localhost
   export POSTGRES_PORT=5432
   ```

3. Apply migrations and seed the database from within `flax-10-project-files/backend`:

   ```
   python manage.py migrate
   python manage.py seed
   ```

Note: The project reads database connection values from environment variables when provided. The default fallback values are `postgres`/`postgres`/`1234`/ `localhost`/`5432`.

Manual Setup Without Nix

This section is intended for developers who want to work without the Nix flake.

Backend

1. Create and activate a virtual environment from the repository root:

   ```
   python3.12 -m venv flax-10-project-files/venv
   source flax-10-project-files/venv/bin/activate
   ```

2. Install Python dependencies:

   ```
   pip install -r requirements.txt
   ```

3. Configure PostgreSQL as described in Chapter 5.

4. Run migrations and seed data:

   ```
   cd flax-10-project-files/backend
   python manage.py migrate
   python manage.py seed
   ```

Frontend

1. Install frontend dependencies:

   ```
   cd flax-10-project-files/frontend
   npm install
   ```

2. If required, provide frontend environment variables before starting Vite. The application can run without optional third-party API keys, but related map and travel-time features depend on them.

Running the Application

Backend (Django)

From within `flax-10-project-files/backend`:

```
python manage.py runserver
```

The API will be available at `http://localhost:8000`.

Frontend (React / Vite)

From within `flax-10-project-files/frontend`:

```
npm run dev
```

The frontend will be available at `http://localhost:5173` (or the port shown in the Vite terminal output).

Note: When using the Nix workflow, start both servers with `nix run .#run`. That command selects the first available ports starting from `8000` for the backend and `5173` for the frontend.

Running the Tests

Backend Tests and Coverage

From within `flax-10-project-files/backend`:

```
python -m coverage erase
python -m coverage run --source=studyFlax manage.py test --verbosity=1
python -m coverage report -m
python -m coverage html
```

The HTML coverage report is generated at `flax-10-project-files/backend/htmlcov/index.html` when run manually.

Note: The Nix test entrypoint writes backend coverage to `coverage/backend/html/index.html` and `coverage/backend/coverage.xml`.

Frontend Tests and Coverage

From within `flax-10-project-files/frontend`:

```
npm run test
npm run coverage
```

When using the Nix test entrypoint, frontend coverage is copied to `coverage/frontend/`.

Environment Variables and Third-Party Services

Backend Variables

Variable	Required	Purpose
POSTGRES_DB	Yes	PostgreSQL database name
POSTGRES_USER	Yes	PostgreSQL username
POSTGRES_PASSWORD	Yes	PostgreSQL password
POSTGRES_HOST	Yes	PostgreSQL host
POSTGRES_PORT	Yes	PostgreSQL port
FRONTEND_URL	Recommended	Used in password reset links
OPENAI_API_KEY	Optional	Enables AI analytics features

Frontend Variables

Variable	Required	Purpose
VITE_API_BASE_URL	Recommended	Backend API base URL
VITE_MAPBOX_TOKEN	Optional	Mapbox-powered map and geocoding features
VITE_TFL_API_KEY	Optional	TfL travel-time integration

Third-Party Services

- **Mapbox** — required for campus map and location autocomplete features. Provide a valid `VITE_MAPBOX_TOKEN`.
- **TfL API** — required for travel-time features. Provide a valid `VITE_TFL_API_KEY`.
- **OpenAI API** — optional. AI analytics endpoints are only fully functional when `OPENAI_API_KEY` is set.
- **SMTP email** — password reset emails use the SMTP configuration currently present in `settings.py`. For any real deployment, replace the default email account details with deployment-specific credentials.

User Fixtures and Access Credentials

The following fixture accounts are available after seeding the database. All passwords are `1234`.

Username	Role	Password
johndoe	Student / Admin	1234
janedoe	Student	1234
amiller	Teacher	1234
charlie	Student	1234

Additional users are seeded automatically using Faker.

Django admin panel: `http://localhost:8000/admin/`

Signing in as `johndoe` with password `1234` also grants access to admin features through the main application interface.

Deployed System

The production deployment is hosted on Railway:

https://flax-10-production.up.railway.app/

Deployment Notes

The repository includes a top-level `Dockerfile` that builds the frontend and backend into a single container image. The container build accepts the following build arguments for the frontend:

```
VITE_MAPBOX_TOKEN
VITE_TFL_API_KEY
OPENAI_API_KEY
```

For deployment, ensure that the runtime environment also provides the backend database connection variables (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_HOST`, `POSTGRES_PORT`) and `FRONTEND_URL` if password reset emails should point to the deployed frontend rather than localhost.

Suggested deployment procedure for IT staff:

1. Provision a PostgreSQL database.
2. Set the backend environment variables listed in Chapter 9.
3. Provide the frontend build arguments required by the enabled third-party features.
4. Build the Docker image using the repository `Dockerfile`.
5. Run Django migrations:

   ```
   python manage.py migrate
   ```

6. Optionally seed demonstration data:

   ```
   python manage.py seed
   ```

Troubleshooting

Nix experimental features error.

> If you encounter an error relating to `nix-command` or `flakes`, ensure the flags are passed as shown in Chapter 4, or add them permanently to `/etc/nix/nix.conf`.

Database connection error.

> Verify that your PostgreSQL server is running and that the `POSTGRES_*` environment variables match the target database. For non-Nix local setup, the fallback defaults are `postgres`/`postgres`/`1234`/ `localhost`/`5432`.

Port already in use.

> If `localhost:8000` or `localhost:5173` is already occupied, terminate the conflicting process or start the server on an alternative port:
>
> ```
> python manage.py runserver 8080
> ```
>
> For the frontend, you can use:

```
npm run dev -- --port 5174
```

Coverage module not found.

> Ensure `coverage` is installed in the virtual environment:
>
> ```
> pip install coverage
> ```

Re-seeding inflates data.

> Running `python manage.py seed` repeatedly may duplicate seed data. To reset the database to a clean state, run the following from within `flax-10-project-files/backend`:
>
> ```
> python manage.py flush --no-input
> python manage.py migrate
> python manage.py seed
> ```
>
> If you are using the Nix workflow, `nix run .#unseed` can also be used to remove seeded data before running `nix run .#seed`.

Package Summary

Backend

Package	Version	Purpose
Django	5.2.0	Web framework
djangorestframework	3.16.1	REST API framework
psycopg	3.3.2	PostgreSQL driver
psycopg-binary	3.3.2	Precompiled psycopg binaries
faker	40.4.0	Fake data generation for seeding
python-dotenv	1.2.1	Environment variable management
openai	2.21.0	OpenAI API integration
httpx	0.28.1	HTTP client library
whitenoise	6.11.0	Static file serving in deployment

Frontend

Package	Version	Purpose
react	19.2.0	UI library
react-dom	19.2.0	React DOM renderer
vite	7.2.4	Build tool and dev server
framer-motion	12.34.3	Animation library
mapbox-gl	3.18.1	Map rendering
styled-components	6.3.9	Component-scoped styling
eslint	9.39.1	JavaScript linter

Package Management

Tool	Version	Purpose
Nix	2.33.2	Environment package management
npm	10.8.2	JavaScript package management